Reading group notes: KV cache compression

Reading group of the Ashbury machine learning lab

1 Background

Most prior methods treat the prefill stage as fixed and tune the cache page around it. This work studies KV cache compression, a method that rethinks the quantization group. Despite steady progress, the interaction between the attention sink and the eviction score is still poorly understood. We argue that the quantization group deserves as much attention as the prefill stage.

The prefill stage also makes KV cache compression easier to inspect, since its behaviour can be read off per token. A limitation of KV cache compression is that the decoding step must be tuned separately for each model size. We expect the cache page to interact with the attention sink at larger scales, which we leave to future work. The results suggest that the attention sink is the main lever for quality at a fixed budget.

A practical system has to balance the cost of the attention sink against the quality of the head dimension. Despite steady progress, the interaction between the decoding step and the eviction score is still poorly understood. Practitioners report that the quantization group is the first component to fail on long inputs. Most prior methods treat the prefill stage as fixed and tune the cache budget around it.

2 Methods we read

Each block of KV cache compression applies the cache page after a short convolution of width 7. During inference KV cache compression caches the head dimension, so each new token costs a constant amount of work. Unlike per-token quantization, KV cache compression updates the attention sink only when the cache page changes. We implement the prefill stage as a fused kernel, which avoids writing intermediate results to memory.

The cache page in KV cache compression is parameterised by a small linear layer with 32 hidden units. We implement the key-value cache as a fused kernel, which avoids writing intermediate results to memory. A coefficient controls how strongly the prefill stage depends on the cache page; we set it to 0.8 throughout. During inference KV cache compression caches the cache page, so each new token costs a constant amount of work.

A coefficient controls how strongly the attention sink depends on the cache page; we set it to 0.5 throughout. KV cache compression computes the eviction score once per layer and reuses it for every head. Gradients through the cache page are clipped at a norm of 9.0. Unlike a recent-window policy, KV cache compression updates the decoding step only when the key-value cache changes.

3 Results we discussed

We train every model for 50B tokens with the same data order and report the mean of three seeds. The gains of KV cache compression grow with the input length, as the head dimension becomes the bottleneck. Variance across seeds stays below 0.6 points for every configuration we report. Ablating the cache budget lowers the RULER-Synth score by 4.9 points, the largest drop among all components.

On NeedleBench, KV cache compression improves over per-token quantization by 6.3 points. The gains of KV cache compression grow with the input length, as the cache page becomes the bottleneck. Variance across seeds stays below 0.5 points for every configuration we report. With the attention sink frozen, KV cache compression still recovers most of its improvement on LongSuite.

We train every model for 50B tokens with the same data order and report the mean of three seeds. KV cache compression scores 47.5% on NeedleBench, compared with 74.5% for a recent-window policy. Throughput is measured with a batch size of 16 on a single accelerator. Variance across seeds stays below 0.2 points for every configuration we report.

4 Open questions

A limitation of KV cache compression is that the head dimension must be tuned separately for each model size. Our analysis covers models up to 1.3B parameters, and larger models may behave differently. The cache budget also makes KV cache compression easier to inspect, since its behaviour can be read off per token. The results suggest that the prefill stage is the main lever for quality at a fixed budget.

Combining KV cache compression with methods that target the key-value cache is a promising direction. Our analysis covers models up to 1.3B parameters, and larger models may behave differently. The cache page also makes KV cache compression easier to inspect, since its behaviour can be read off per token. A limitation of KV cache compression is that the prefill stage must be tuned separately for each model size.